Introduction to Machine Learning Methods

Unsupervised learning

Firooz Bashashi Saghezchi
RWTH Aachen University

Outline

- Unsupervised learning
  - Clustering Algorithms
    - K-means Clustering
    - Expectation Maximization

Reference:

Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006. (Chapter 9)

Available online: Pattern Recognition and Machine Learning

Introduction to clustering

Different Types of Machine Learning Algorithms

- **Supervised Learning:** requires labeled data $\mathbf{x}^{(i)} = \left(x_1^{(i)}, \dots, x_n^{(i)}; y^{(i)}\right); i = 1, \dots, m$
- **Unsupervised Learning:** relies on unlabeled data $\mathbf{x}^{(i)} = \left(x_1^{(i)}, \dots, x_n^{(i)}\right); i = 1, \dots, m$
- **Semi-supervised Learning:** The dataset contains mostly of unlabeled data with only few labeled examples for model calibration
- **Reinforcement Learning:** There is no a priori training dataset; the model learns from its experiences (interactions with the environment)

Different Types of Machine Learning Algorithms

- **Supervised Learning:** requires labeled data $\mathbf{x}^{(i)} = \left(x_1^{(i)}, \dots, x_n^{(i)}; y^{(i)}\right); i = 1, \dots, m$
- **Unsupervised Learning:** relies on unlabeled data $\mathbf{x}^{(i)} = \left(x_1^{(i)}, \dots, x_n^{(i)}\right); i = 1, \dots, m$
- **Semi-supervised Learning:** The dataset contains mostly of unlabeled data with only few labeled examples for model calibration
- **Reinforcement Learning:** There is no a priori training dataset; the model learns from its experiences (interactions with the environment)

In this lecture, we focus on **Unsupervised Learning**

Applications of Unsupervised Learning

- **Data mining** (discovering useful knowledge from data)
- **Pattern recognition** (detecting and classifying patterns)
- **Image segmentation** (dividing an image into meaningful regions)
- **Bioinformatics** (organizing biological data into meaningful groups based on genes, DNA, cells, or patients)
- **Customer segmentation** (grouping customers with similar characteristics together)
- **Social network segmentation** (detecting how users connect, interact, or behave)
- **Anomaly detection**
- **Feature reduction**

Unsupervised Learning Methods

- **Clustering:** Grouping similar examples together (and dissimilar ones far apart)
- **Anomaly Detection:** Determining the probability density of the benign examples to set bounties for anomaly (outlier) detection
- **Dimensionality Reduction:** Eliminating redundant or less informative features from training/test data before constructing a machine learning model

Data Clustering

- Clustering is a technique for finding similar groups in data, called clusters
- It is an unsupervised learning method, i.e., it does not require ground truth labels ($y^{(i)}$s)
- The objective is to group data instances that are similar to (near) each other in one cluster and the ones that are very different (far away) into different clusters

Given m unlabelled examples $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots, \mathbf{x}^{(m)} \in \mathbb{R}^n$ and the desired number of partitions k, group data into k "homogeneous" partitions

Distance/Similarity Metric

- Clustering only looks at similarities between input features $\mathbf{x}^{(i)}$s – no ground truth label ($y^{(i)}$) is given
- However, without ground truth labels, similarity can be hard to define
- The definition of similarity is subjective; one can use different metrics to define it

The choice of similarity metric impacts the output clusters

Distance/Similarity Metric

- We often use **Euclidian distance (L2 norm or norm 2)** as our distance metric
- For a given training example $\mathbf{x}^{(i)} = \left(x_1^{(i)}, \ldots, x_n^{(i)}\right)$, Euclidean distance is defined as:

$$\left\|\mathbf{x}^{(i)}\right\|_2 = \left\|\mathbf{x}^{(i)}\right\| = \sqrt[2]{\left(x_1^{(i)}\right)^2 + \cdots + \left(x_n^{(i)}\right)^2}$$

The use of subscript 2 is optional for Euclidean distance

Without subscript by default means norm 2

Different Similarity Metrics (Norms)

- Depending on the application, we may use others norms as distance metrics
- **Manhattan (city block) distance** (or **norm 1**):

$$\left\|\mathbf{x}^{(i)}\right\|_1 = \left|x_1^{(i)}\right| + \cdots + \left|x_n^{(i)}\right|$$

- **Norm-p distance**:

$$\left\|\mathbf{x}^{(i)}\right\|_p = \sqrt[p]{\left(x_1^{(i)}\right)^p + \cdots + \left(x_n^{(i)}\right)^p}$$

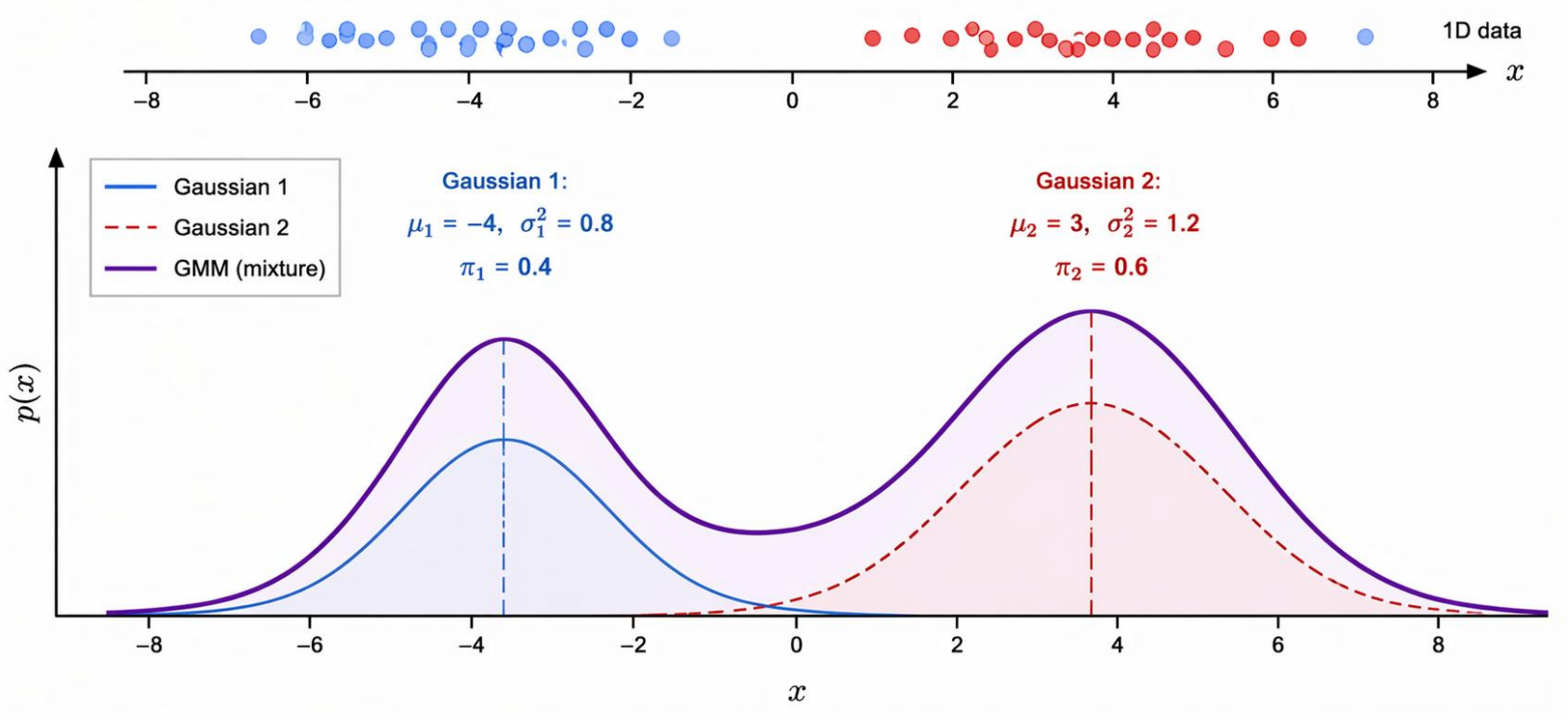
- As an special case, when $p = \infty$, we end up with **infinity (maximum) norm**:

$$\left\|\mathbf{x}^{(i)}\right\|_\infty = \max\left(\left|x_1^{(i)}\right|, \ldots, \left|x_n^{(i)}\right|\right)$$

Different Clustering Algorithms

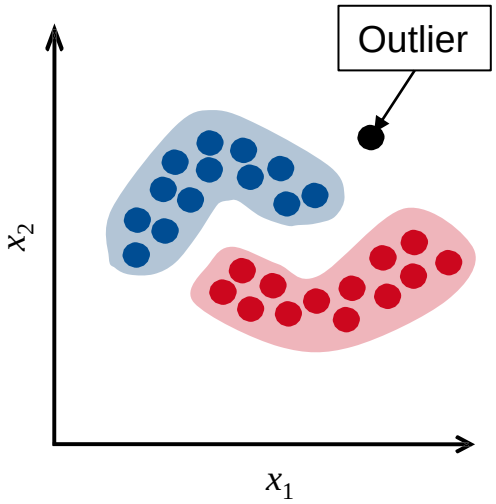
- **Distribution-based clustering** (sometimes called *probabilistic clustering*) groups data points according to their underlying probability distributions
- A commonly used algorithm for this approach is the Expectation-Maximization (EM) algorithm
- It assumes that the data are generated from a **Gaussian Mixture Model (GMM)**, i.e., a mixture of multiple Gaussian distributions
- In this model, the **mean** of each Gaussian represents the cluster centroid, while the **standard deviation (or covariance)** determines the spread (size and shape) of the cluster

Gaussian Mixture Model (GMM)



Different Clustering Algorithms (Continued)

- **Density-based** clustering identifies regions where data points are densely concentrated and separates them from regions that are sparse or empty
- Unlike the previous two methods, it can detect **clusters with arbitrary shapes**
- A well-known algorithm in this category is Density-Based Spatial Clustering of Applications with Noise (DBSCAN)
  - ε: radius around a point defines its “neighborhood”
  - P: minimum number of points to form a dense region (e.g. $P = 5$)



Hard versus Soft Clustering

- **Hard clustering:** a given data point is always assigned to a single cluster
- **Examples of hard clustering algorithms:** k-means and DBSCAN

Hard clustering example with two clusters

Data point $\mathbf{x}^{(i)}$	Assigned Cluster $y^{(i)}$
$\mathbf{x}^{(1)}$	1
$\mathbf{x}^{(2)}$	2
$\mathbf{x}^{(3)}$	2
$\mathbf{x}^{(4)}$	1

Hard versus Soft Clustering

- **Soft Clustering:** a given data point can be assigned to different clusters with different probabilities (*responsibilities*): $r_1^{(i)} = p\{y^{(i)} = 1\}, \ldots, r_K^{(i)} = p\{y^{(i)} = K\}$

$$r_1^{(i)} + \cdots + r_K^{(i)} = 1$$

$r_k^{(i)}$ is the responsibility of cluster k for example i

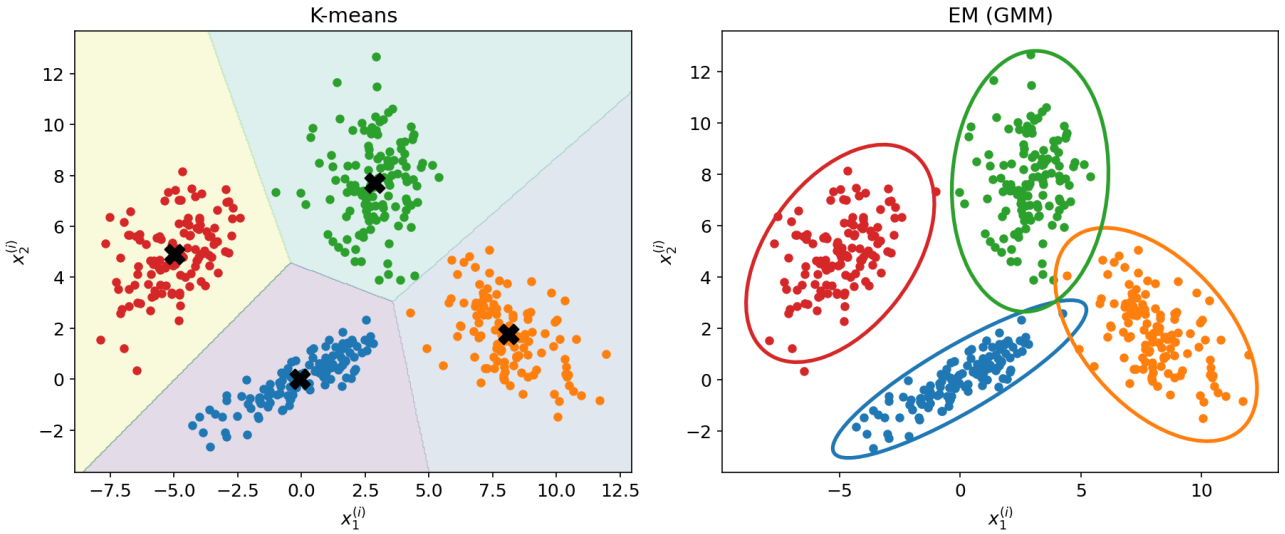
- **Example of soft clustering algorithm:** EM

Soft clustering example with two clusters

Data point $\mathbf{x}^{(i)}$	$r_1^{(i)}$	$r_2^{(i)}$
$\mathbf{x}^{(1)}$	0.5	0.5
$\mathbf{x}^{(2)}$	0.2	0.8
$\mathbf{x}^{(3)}$	0.7	0.3
$\mathbf{x}^{(4)}$	0.4	0.6

Flat Clustering

- Flat clustering divides data into a fixed number of non-overlapping clusters
- Examples: k-means (circular clusters) and EM (ellipsoidal clusters)
- Applicable to relatively large datasets (computationally efficient)



Flat Clustering (Limitations)

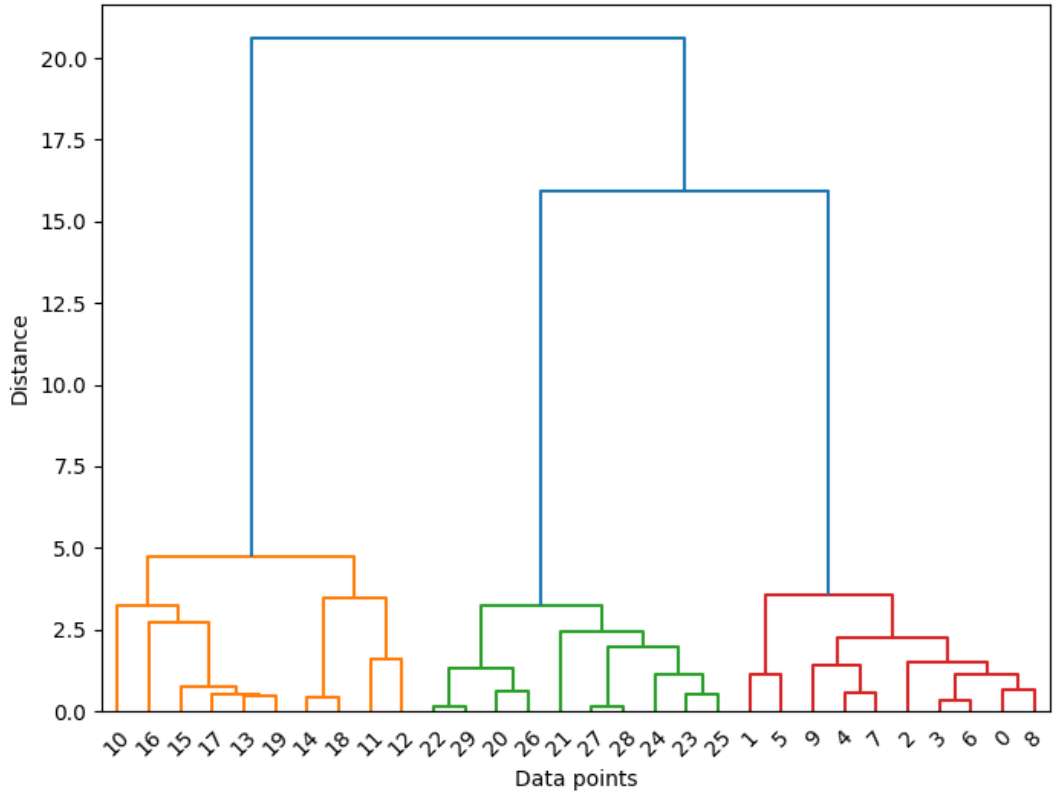
- Sensitive to the choice of initial centroids
- Heuristics are used to select good initial centroids (e.g., choosing centroids that are far apart)
- May converge to a local optimum
- The choice of the number of clusters (K) is subjective

Hierarchical Clustering

- Builds a hierarchy (tree) of clusters
- Two approaches: bottom-up (**agglomerative**) or top-down (**divisive**) clustering

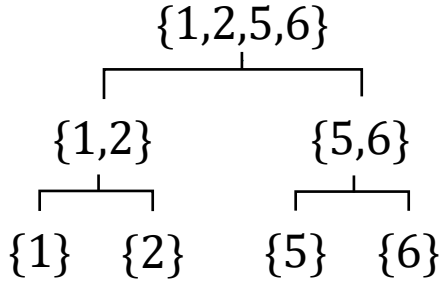
1. **Agglomerative (bottom-up):**
   - Starts with M clusters (one data point per cluster)
   - Iteratively merges the most similar clusters
2. **Divisive (top-down):**
   - Starts with a single cluster containing all data points
   - Recursively splits clusters into smaller ones

Hierarchical Clustering (Dendrogram)



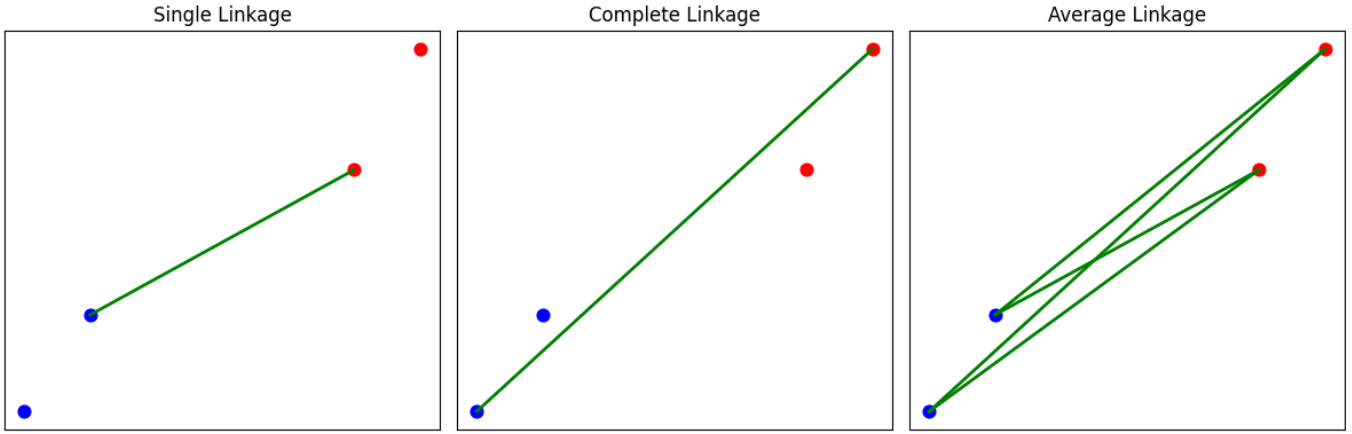
Hierarchical Clustering

- It is a heuristic clustering algorithm (there is no explicit global objective function)
- Produces a full clustering hierarchy (not just one partition)
- Always returns a clustering structure, even if the data has no pattern at all
- Takes an $M \times M$ **dissimilarity matrix,** distance between all data points, as input
- Returns clustering solution for every number of clusters 1,…,K
- For example: suppose we have the following dataset: $\{1,2,5,6\}$
- However, it has high computational complexity



Hierarchical Clustering – Linkage Metric

- Requires a criterion (e.g., **single, complete, average linkage**) to measure separation (distance) between two clusters
- Singe (complete) linkage is defined as the distance between the two nearest (farthest) points of two clusters, while average linkage is the average distance between all their members



K-means algorithm

K-means Algorithm

- Input:
  - A set of unlabeled training examples $\{\mathbf{x}^{(i)}, i = 1, \dots, m\}$; $\mathbf{x}^{(i)} \in \mathbb{R}^n$;
  - Number of clusters for partitioning data: K

- Output:
  - Cluster assignment $y^{(i)}$ for every data point $\mathbf{x}^{(i)}$
  - K centroids $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K \in \mathbb{R}^n$

K-means Algorithm

- Randomly initialize K centroids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K \in \mathbb{R}^n$
- Repeat until convergence:
  - For every datapoint $\mathbf{x}^{(i)}$, (re)assign it to the nearest cluster:

Assigned cluster for $\mathbf{x}^{(i)}$ *Centroid of cluster k*

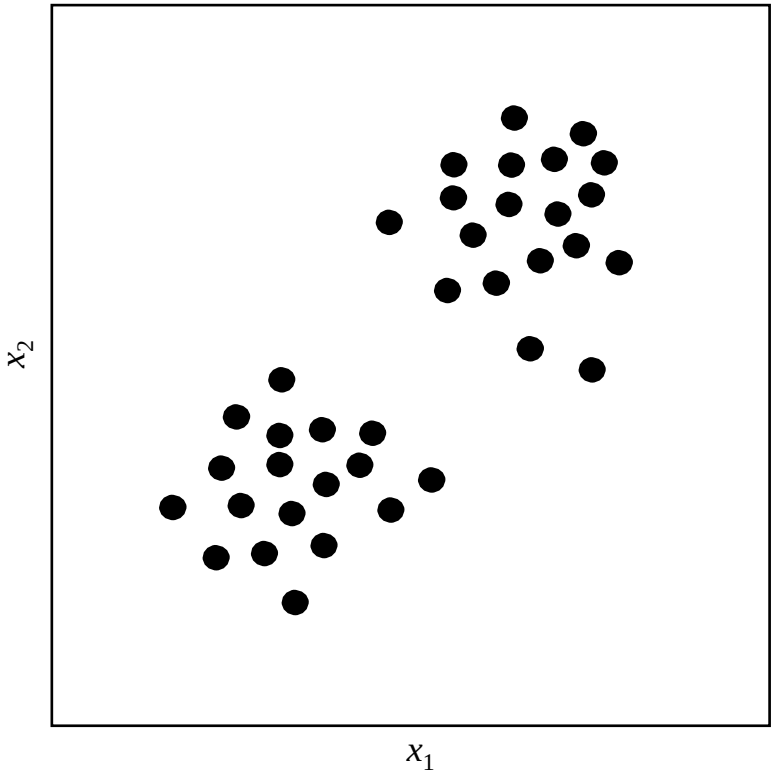
$$y^{(i)} = \arg\min_k \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_k \right\|^2$$

  - For every cluster k, update its centroid to the mean of its members:

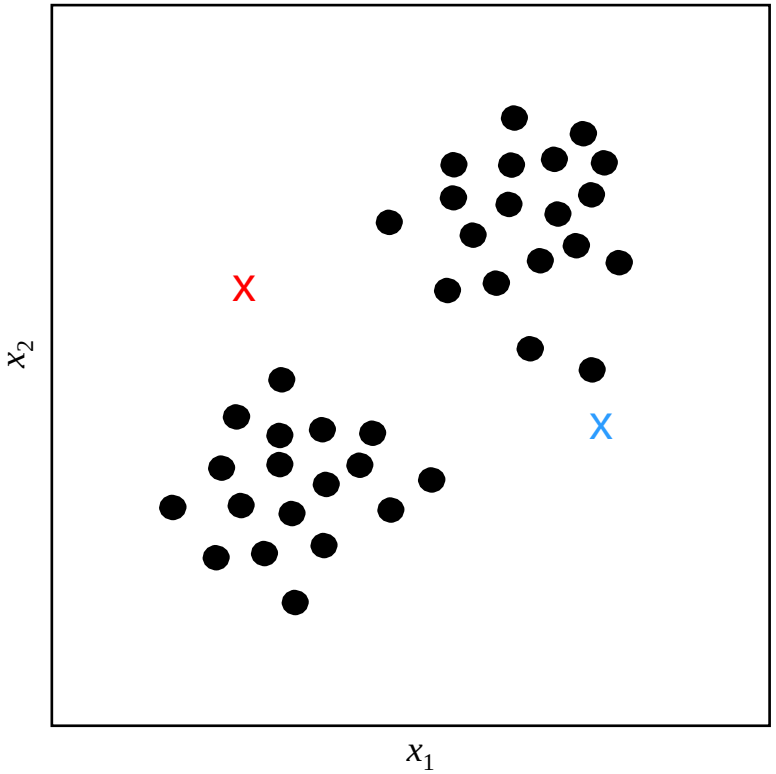
$$\boldsymbol{\mu}_k = \frac{\sum_{i=1}^{m} 1\left(y^{(i)} = k\right)\mathbf{x}^{(i)}}{\sum_{i=1}^{m} 1\left(y^{(i)} = k\right)}$$

  - Here, $1(y^{(i)} = k)$ is the indicator function. It returns one when its argument is true ($y^{(i)} = k$), otherwise (if $y^{(i)} \neq k$), it returns zero
- The algorithm converges when the centroids stop changing

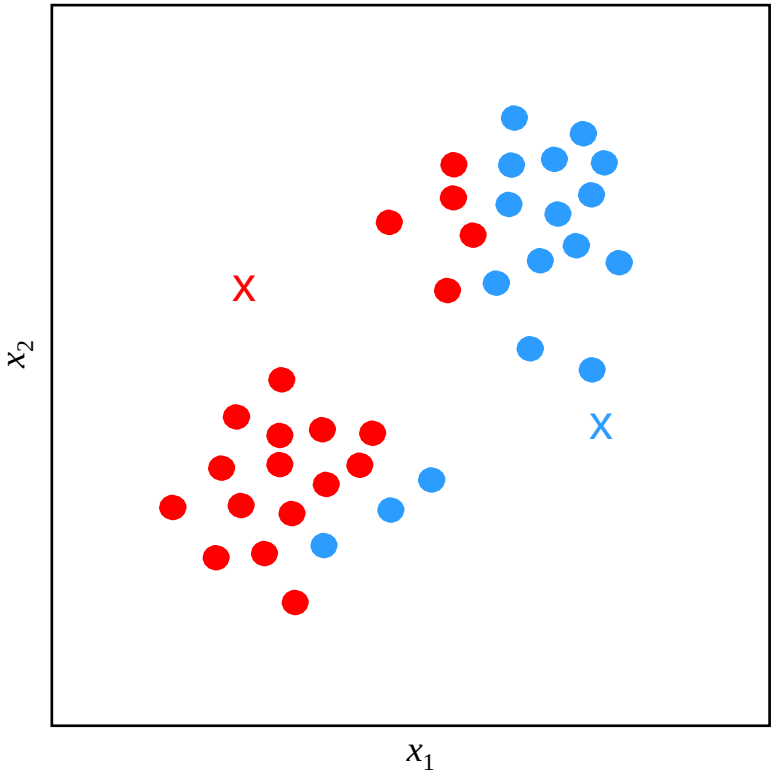
K-means Algorithm – Example



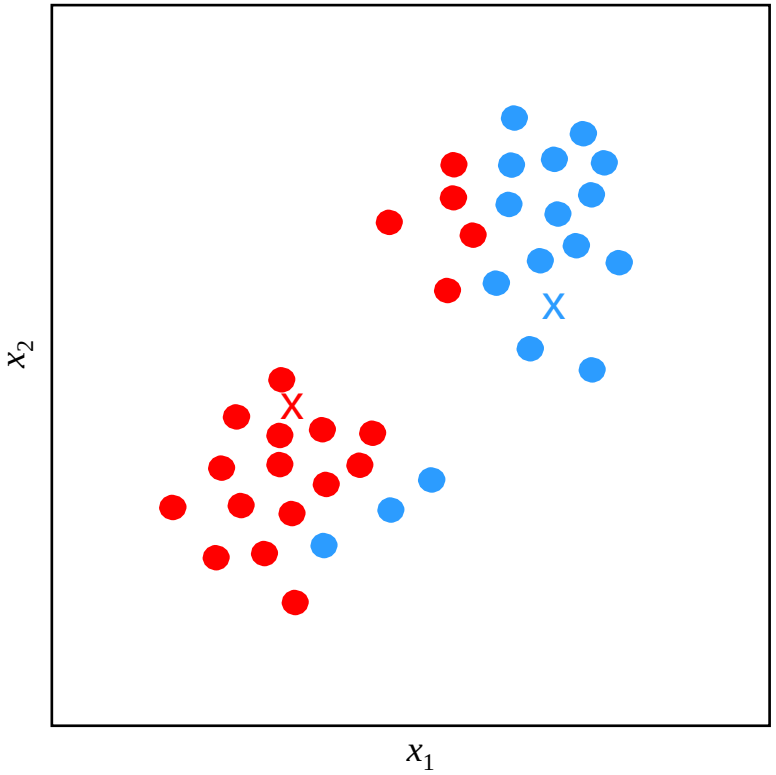
K-means Algorithm – Centroids Initialization



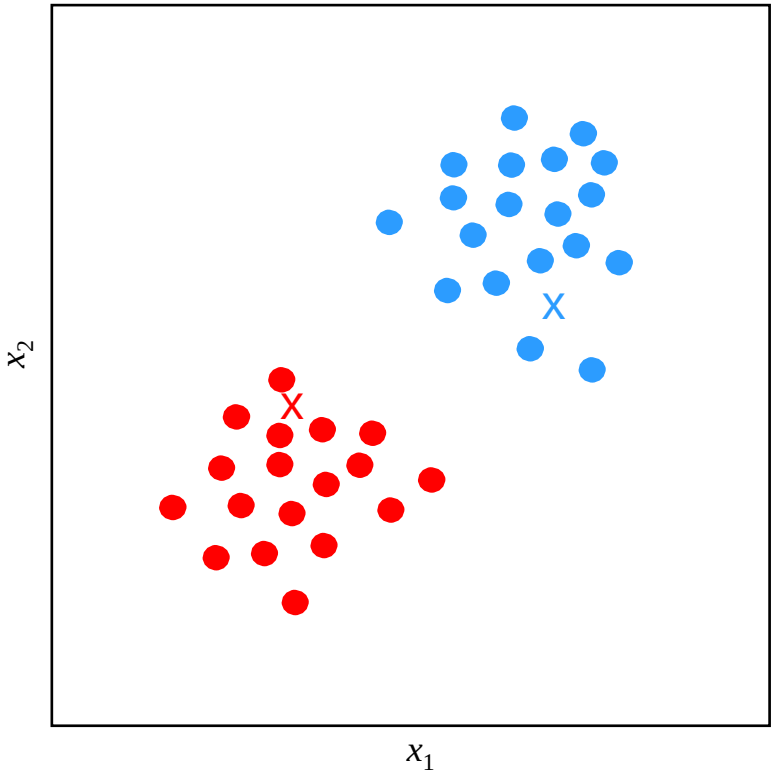
K-means Algorithm – Assignments to Clusters



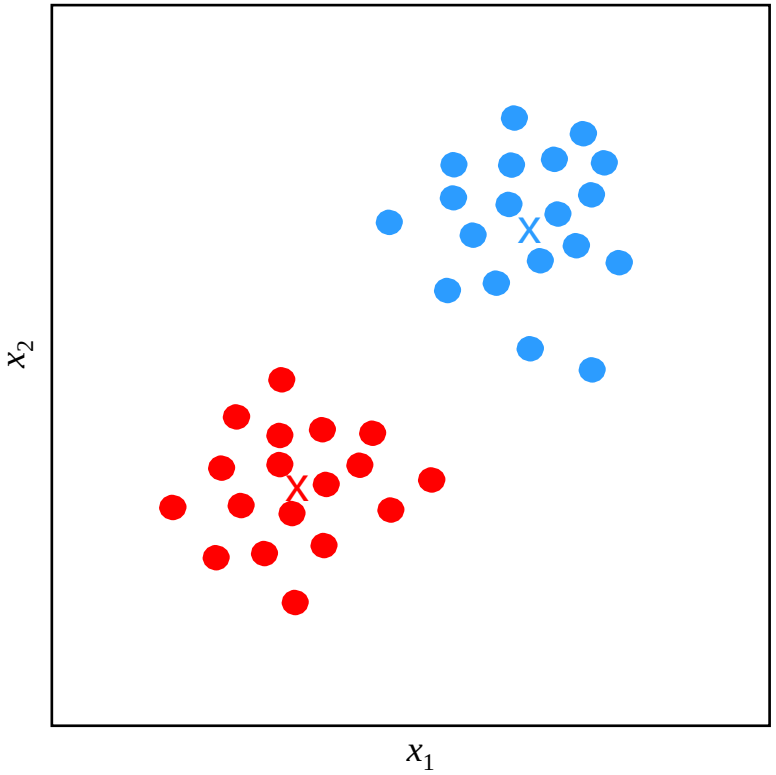
K-means Algorithm – Updating Centroids



K-means Algorithm – Updating Assignments



K-means Algorithm – Updating Centroids



Expectation Maximization (EM) algorithm

Expectation Maximization (EM) Algorithm for Clustering

- Instead of hard assignments (like k-means), EM is a soft assignment algorithm, i.e., each point belongs to each cluster with a probability (responsibility)

$$r_k^{(i)} = p\{y^{(i)} = k\}$$

- EM algorithm has three steps:
  - **Step 1:** Initialization
  - **Step 2:** Expectation (E-Step)
  - **Step 3:** Maximization (M-Step)

Expectation Maximization (EM) Algorithm

- STEP 1: Initialization
  - initialize parameters $\boldsymbol{\theta}_k^0 = (\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k, \pi_k)$
  - means $\boldsymbol{\mu}_k$ (e.g., randomly)
  - covariances $\boldsymbol{\Sigma}_k$ (e.g., $\boldsymbol{\Sigma}_k = \mathbf{I}$)
  - mixing weights π_k (e.g., $\pi_k = \frac{1}{k}$)

Expectation Maximization (EM) Algorithm

- STEP 1: Initialization
  - initialize parameters $\boldsymbol{\theta}_k^0 = (\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k, \pi_k)$
  - means $\boldsymbol{\mu}_k$ (e.g., randomly)
  - covariances $\boldsymbol{\Sigma}_k$ (e.g., $\boldsymbol{\Sigma}_k = \mathbf{I}$)
  - mixing weights π_k (e.g., $\pi_k = \frac{1}{k}$)

- STEP 2: E-Step (Expectation): At every time instance (iteration) t
  - Use current parameters to compute responsibilities, probability that $\mathbf{x}^{(i)}$ belongs to cluster k

$$r_k^{(i)} = \frac{\pi_k \, p\left(\mathbf{x}^{(i)} \middle| \boldsymbol{\theta}_k^{t-1}\right)}{\sum_{k'} \pi_{k'} \, p\left(\mathbf{x}^{(i)} \middle| \boldsymbol{\theta}_{k'}^{t-1}\right)}$$

Expectation Maximization (EM) Algorithm

- STEP 3: M-Step (Maximization): At every time instance (iteration) t
  - Update parameters using calculated responsibilities:

Mixing weights:

$$\pi_k = \frac{\sum_{i=1}^{m} r_k^{(i)}}{m}$$

Expectation Maximization (EM) Algorithm

- STEP 3: M-Step (Maximization): At every time instance (iteration) t
  - Update parameters using calculated responsibilities:

Mixing weights:

$$\pi_k = \frac{\sum_{i=1}^{m} r_k^{(i)}}{m}$$

Means:

$$\boldsymbol{\mu}_k = \frac{\sum_{i=1}^{m} r_k^{(i)} \mathbf{x}^{(i)}}{\sum_{i=1}^{m} r_k^{(i)}}$$

Expectation Maximization (EM) Algorithm

- STEP 3: M-Step (Maximization): At every time instance (iteration) t
  - Update parameters using calculated responsibilities:

Mixing weights:

$$\pi_k = \frac{\sum_{i=1}^{m} r_k^{(i)}}{m}$$

Means:

$$\boldsymbol{\mu}_k = \frac{\sum_{i=1}^{m} r_k^{(i)} \mathbf{x}^{(i)}}{\sum_{i=1}^{m} r_k^{(i)}}$$

Covariances:

$$\boldsymbol{\Sigma}_k = \frac{\sum_{i=1}^{m} r_k^{(i)} \left(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k\right)\left(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k\right)^{\mathrm{T}}}{\sum_{i=1}^{m} r_k^{(i)}}$$

Expectation Maximization (EM) Algorithm

- STEP 3: M-Step (Maximization): At every time instance (iteration) t
  - Update parameters using calculated responsibilities:

Mixing weights:

$$\pi_k = \frac{\sum_{i=1}^{m} r_k^{(i)}}{m}$$

Means:

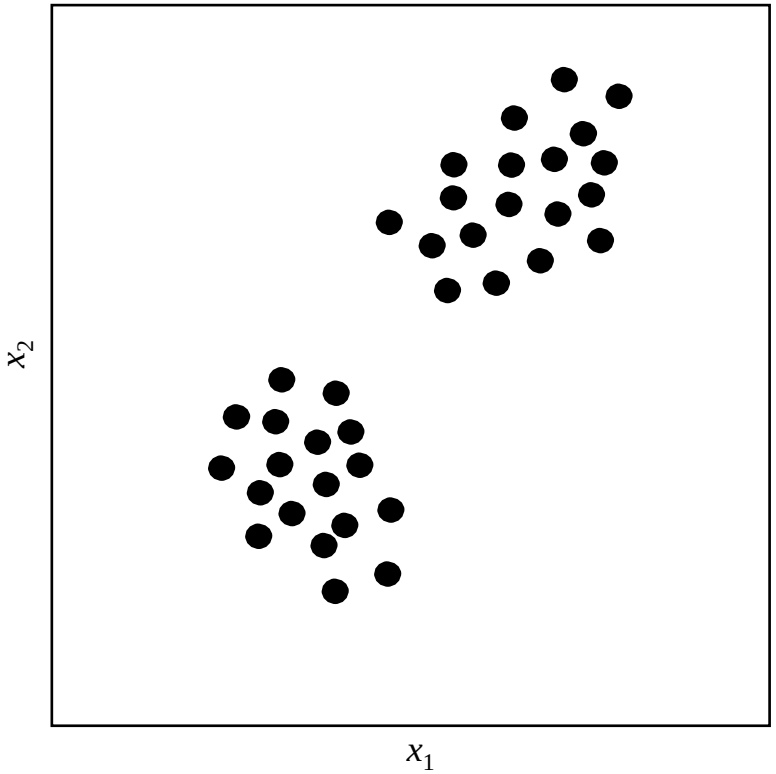
$$\boldsymbol{\mu}_k = \frac{\sum_{i=1}^{m} r_k^{(i)} \mathbf{x}^{(i)}}{\sum_{i=1}^{m} r_k^{(i)}}$$

Covariances:

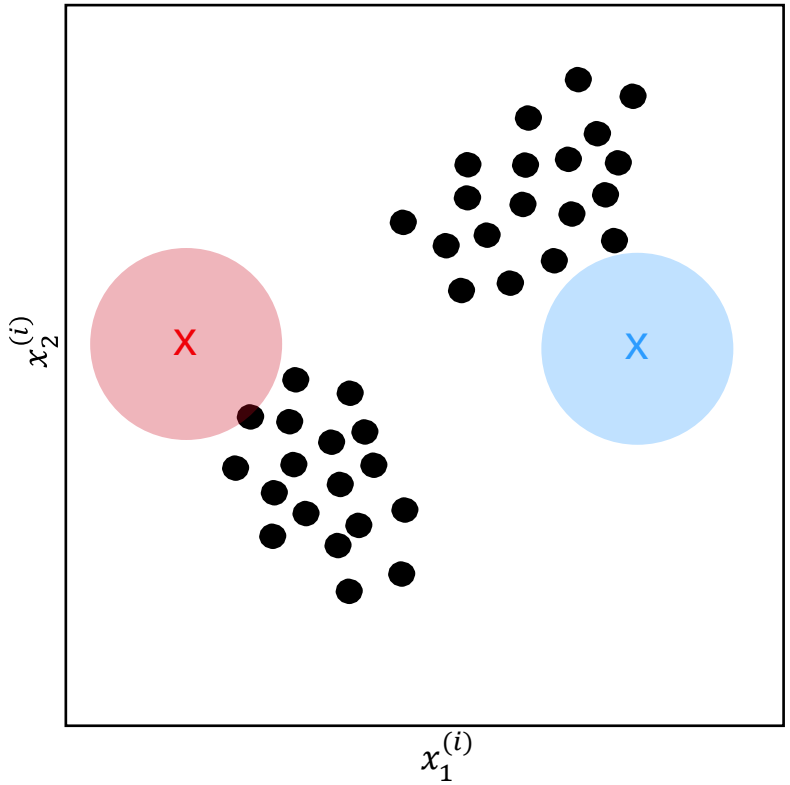
$$\boldsymbol{\Sigma}_k = \frac{\sum_{i=1}^{m} r_k^{(i)} \left(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k\right)\left(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k\right)^{\mathrm{T}}}{\sum_{i=1}^{m} r_k^{(i)}}$$

- If the convergence condition is met, exit; otherwise, go to E-Step (STEP 2)

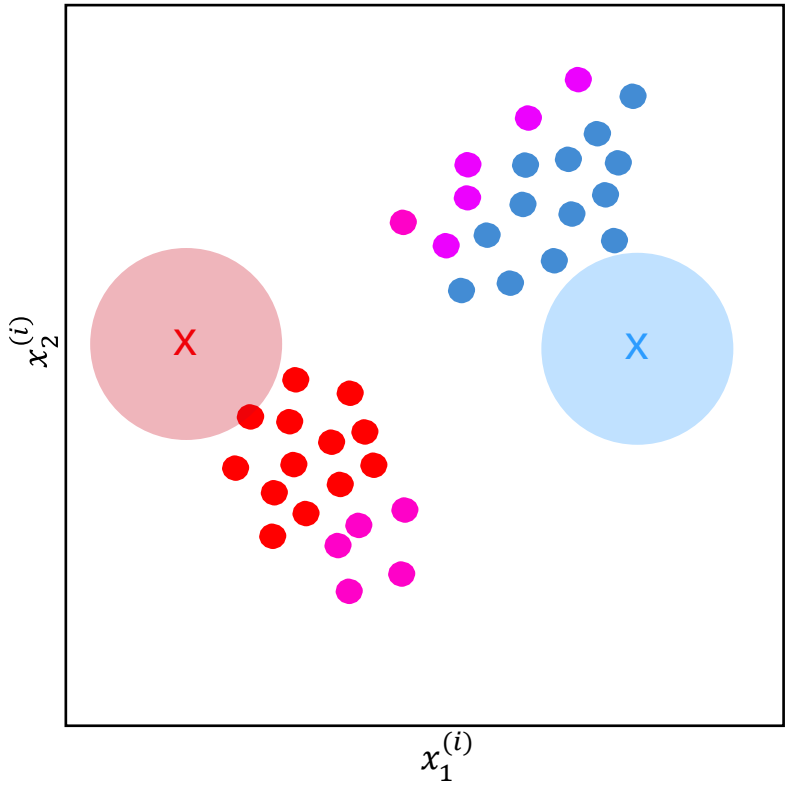
EM Algorithm Example (Dataset)



EM Algorithm – Initialization Step

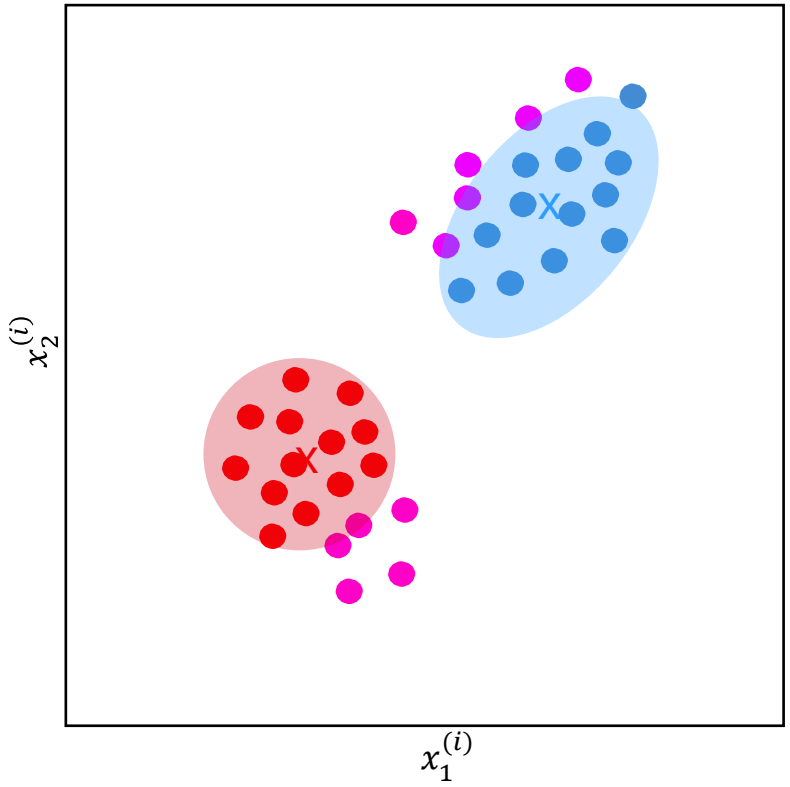


EM Algorithm – Expectation Step



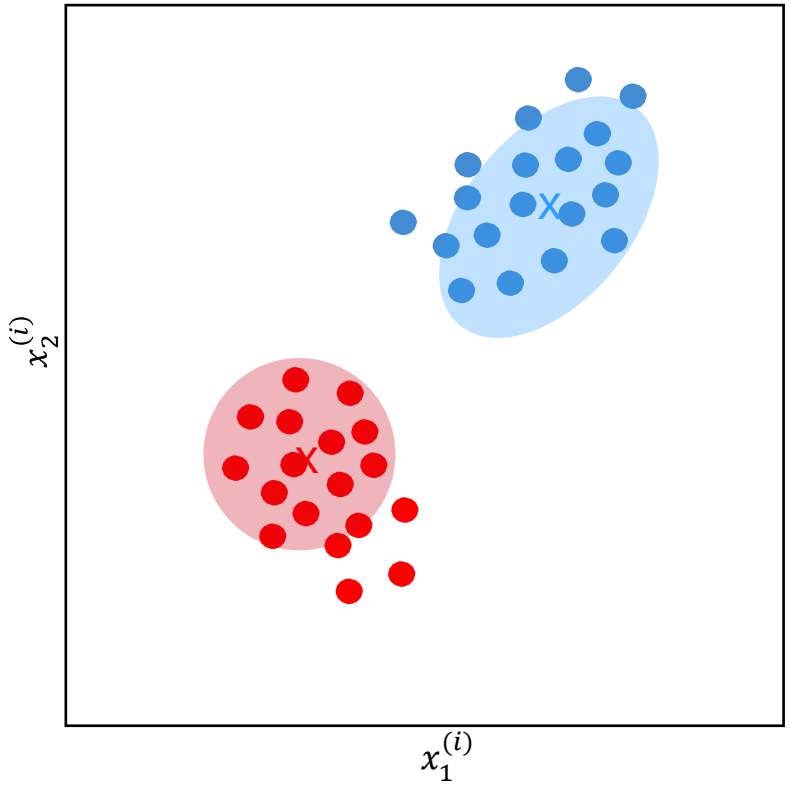
Purple points belong to both clusters

EM Algorithm – Maximization Step

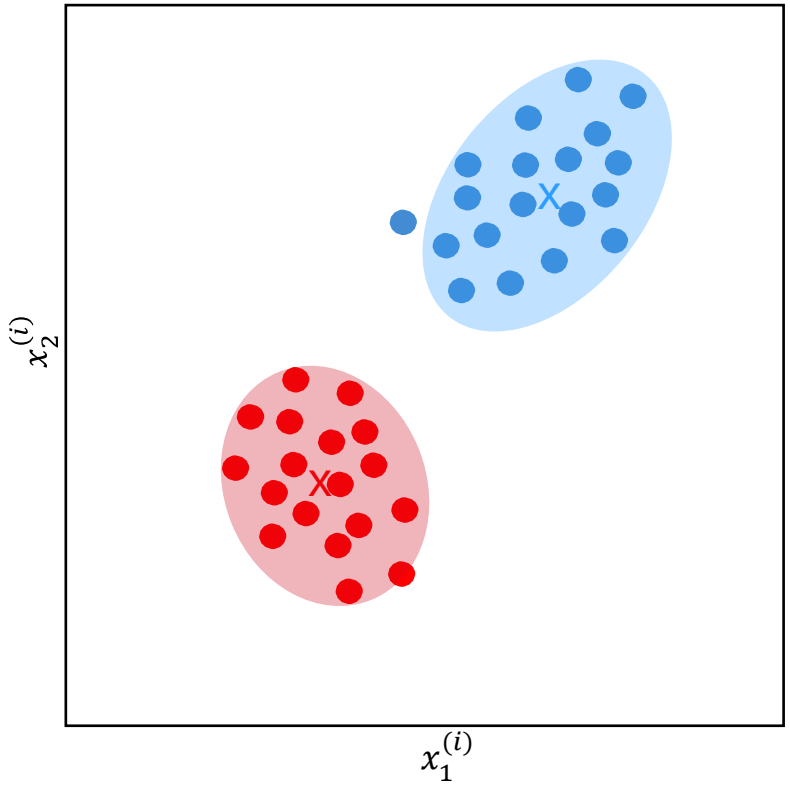


Purple points belong to both clusters

EM Algorithm – Expectation Step



EM Algorithm – Maximization Step



Analogy between K-means and EM

- Note the similarities between K-means and EM algorithm
- Indeed, K-means is helpful when clusters have circular shapes
- EM is more flexible and can fit to ellipsoidal clusters with arbitrary orientations

Metrics to assess clustering algorithms

Evaluation Metrics for Clustering Algorithms

- **Intra-cluster cohesion** measures how near the data points in a cluster are to their centroid – the less, the more compact the cluster

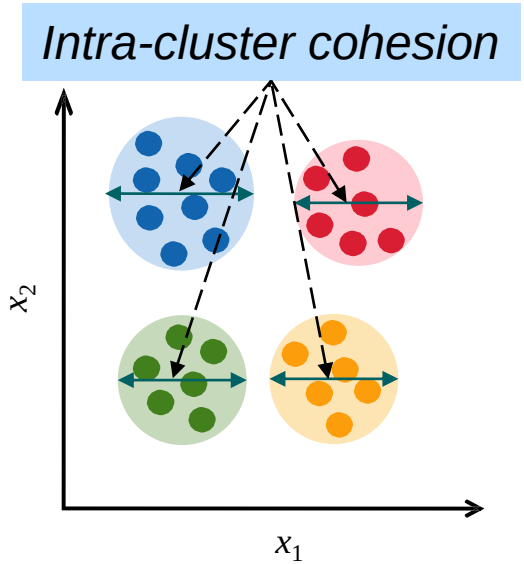
- Mean Squared Error (MSE) can be used as cost metric

- It measures the mean square distance of all cluster members from their respective centroids

$$J = \frac{1}{2m} \sum_{i=1}^{m} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_{y^{(i)}} \right\|^2$$

- $y^{(i)}$ is the assigned cluster for $\mathbf{x}^{(i)}$, and $\boldsymbol{\mu}_{y^{(i)}}$ is its centroid:

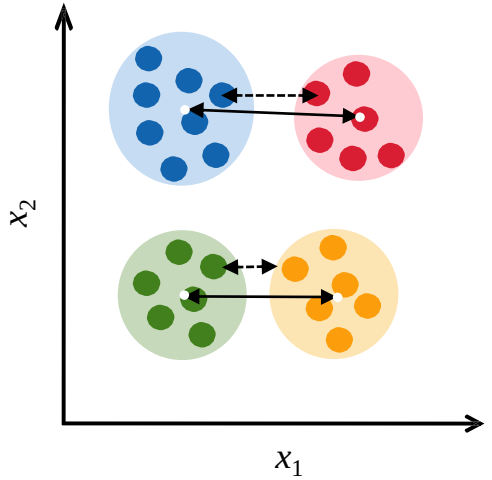
$$y^{(i)} = \arg\min_k \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_k \right\|^2$$



Evaluation Metrics for Clustering Algorithms

- **Inter-cluster separation** measures the distance between clusters
- The distance between two clusters can be measured in different ways, e.g., the distance between their closest points or their respective means or medians
- The larger this metric, the better

Inter-cluster separation



Evaluation Metrics for Clustering Algorithms

- **Silhouette score** is a metric used to assess the overall quality of a clustering algorithm, combining both intra-cluster cohesion and inter-cluster separation

- For a given datapoint $\mathbf{x}^{(i)}$, Silhouette score is defined as:

$$s\left(\mathbf{x}^{(i)}\right) = \frac{b\left(\mathbf{x}^{(i)}\right) - a\left(\mathbf{x}^{(i)}\right)}{\max\{a(\mathbf{x}^{(i)}),\ b(\mathbf{x}^{(i)})\}}$$

$a\left(\mathbf{x}^{(i)}\right)$ is the average distance between $\mathbf{x}^{(i)}$ and all its cluster mates

$b\left(\mathbf{x}^{(i)}\right)$ is the average distance between $\mathbf{x}^{(i)}$ and all members of the nearest neighbor cluster

Overall Silhouette Score

- Dataset level Silhouette score is defined as:

$$S = \frac{1}{2m} \sum_{i=1}^{m} s\left(\mathbf{x}^{(i)}\right)$$

- Interpretation:

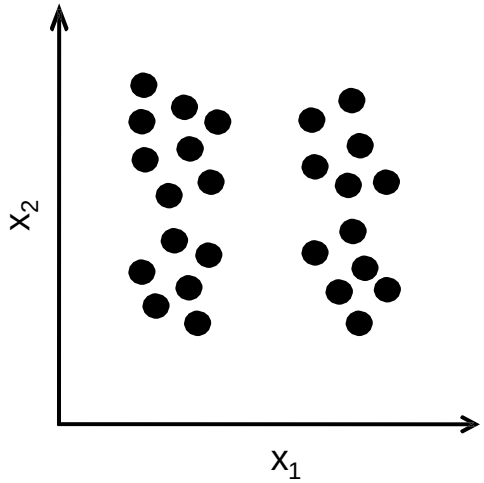
  $S \approx 1$: well-clustered, good separation

  $S \approx 0$: overlapped clusters

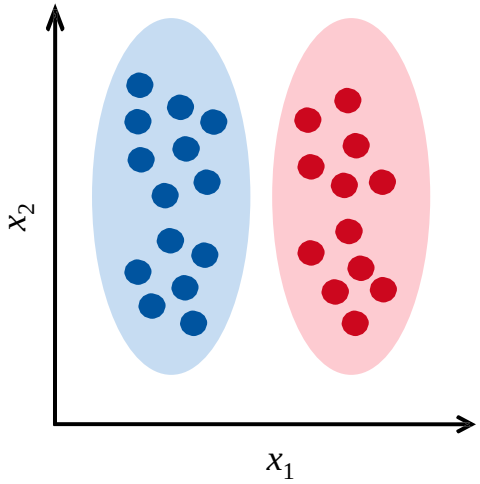
  $S < 0$: likely wrong clustering

How to choose the number of clusters?

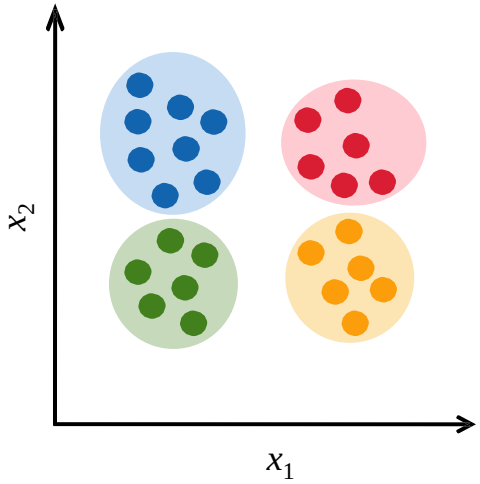
How many clusters do you see?



Two clusters?

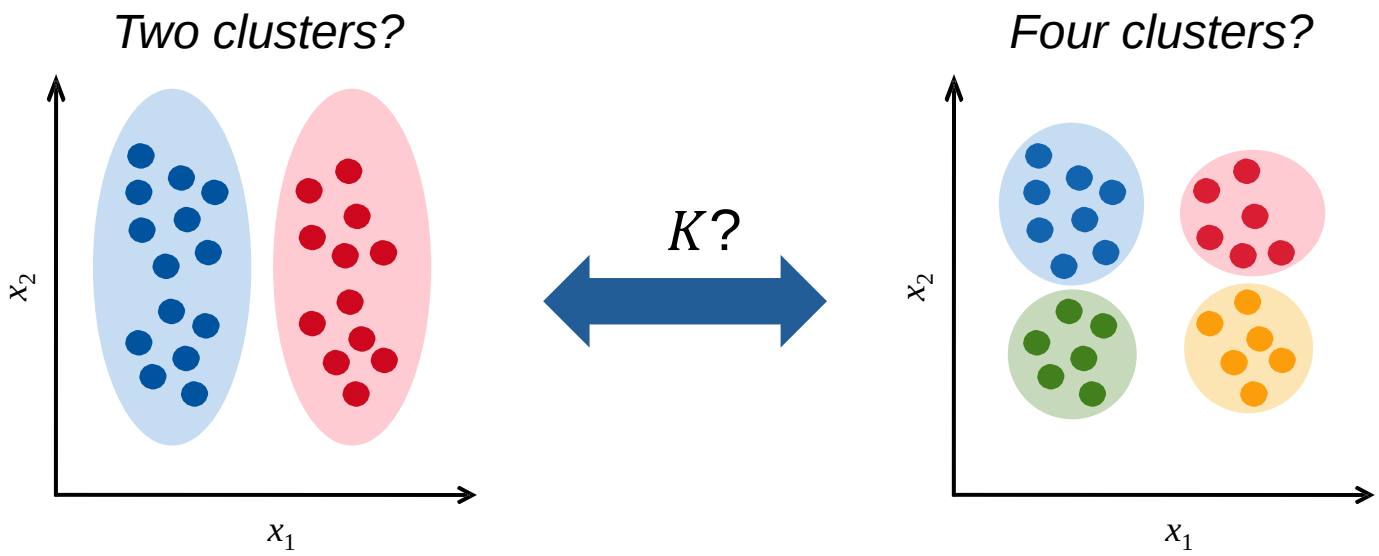


Or, four clusters?

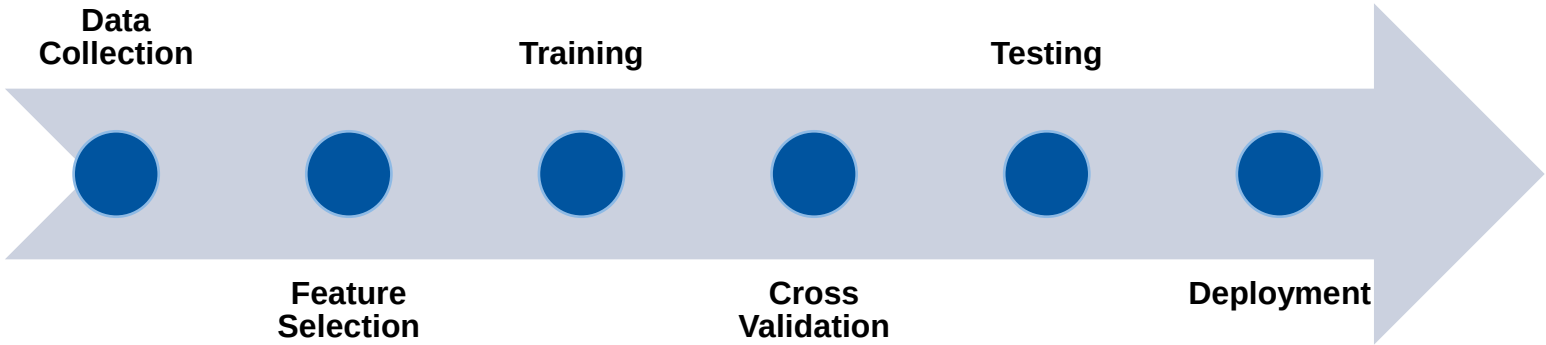


How many clusters do I have?

- We can use any of the metrics define before (distortion or Silhouette score) to answer this question



Process for Training a Machine Learning Model



How to train a machine learning model?

1. Divide your data into three non-overlapping datasets:
   - 65% for **training**
   - 15% for **(cross-)validation**
   - 20% for **testing** (make sure this is statistically independent from both training and validation sets)
2. Choose a hyperparameter (e.g., the number of clusters K)
3. Build the model using the training dataset
4. Assess its performance (loss) over the validation set
5. Repeat steps 3-4 with different hyperparameters and choose the one that performs best (gives the minimum loss) over the validation set
6. Assess the performance of the chosen model using the test dataset

How to determine the number of clusters?

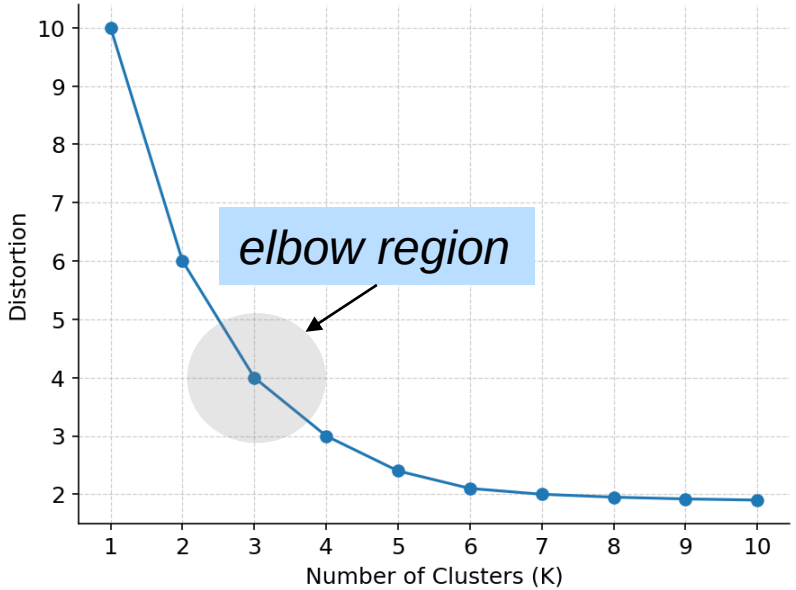
- To assess a clustering algorithm, we may use **MSE (*distortion function*)** as our loss metric:

$$J = \frac{1}{2m} \sum_{i=1}^{m} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_{y^{(i)}} \right\|^2$$

- For example, to determine K, we may try to minimize this loss function
- However, in the extreme case, if we set the number of clusters equal to the number of data points (one object per cluster), J will reach its global minimum (which is zero)
- This is obviously **overfitting** problem

How to determine the number of clusters? (Elbow Point of MSE)

- The distortion function (MSE) decreases monotonically as K increases
- However, it drops fast in the beginning (until the *elbow region*) and afterwards plateaus
- The best choice for K lies in the elbow region, e.g. $K = 3$ in the figure

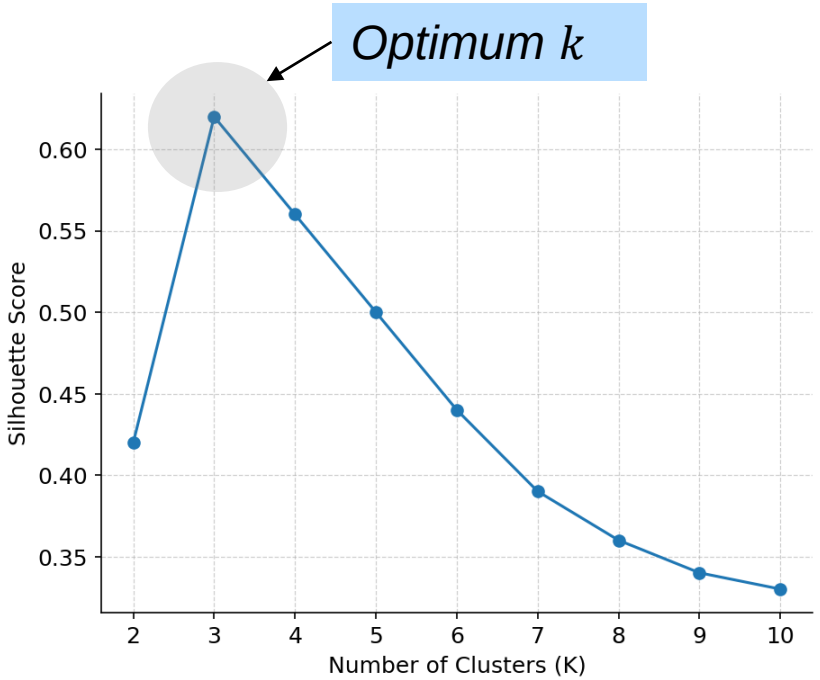


How to determine the number of clusters? (Peak Silhouette Score)

- Alternatively, we may use Silhouette score as our cost metric:

$$S = \frac{1}{2m}\sum_{i=1}^{m}\frac{b\left(\mathbf{x}^{(i)}\right) - a\left(\mathbf{x}^{(i)}\right)}{\max\{a(\mathbf{x}^{(i)}),\ b(\mathbf{x}^{(i)})\}}$$

- We choose the value of K that maximizes the Silhouette score



Caveats for training a K-Means model (Local Optima)

- The distortion function is a **sum of convex functions**, which is non-convex

$$J = \frac{1}{2m} \sum_{i=1}^{m} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_{y^{(i)}} \right\|^2$$

- Hence, for a given K, we may trap into a local minimum
- The same problem can occur with Silhouette score

Caveats for training a K-Means model (Local Optima)

- To avoid this problem, we can run the algorithm with different initializations of centroids
- Each initialization will lead us to a different minimum point of the distortion function, but some of them will hopefully lead us to its global minimum
- For every K, we can run the algorithm for 10-100 different random initialization of centroids and choose the model that yields the minimum distortion (or maximum Silhouette score)

Block Coordinate Decent (cf. Gradient Decent)

- K-means algorithm alternates between two steps:

  1) assigning every datapoint to the nearest cluster

  2) updating the centroids to the mean of their members

- If we look at the distortion function

$$J(y, \boldsymbol{\mu}) = \frac{1}{2m} \sum_{i=1}^{m} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_{y^{(i)}} \right\|^2$$

Block Coordinate Decent (cf. Gradient Decent)

- We realize that the two steps of k-means alternatively minimizes $J(y, \boldsymbol{\mu})$ over its two coordinates:

  - First, it assumes $\boldsymbol{\mu}$ is fixed and finds the assignment of datapoint to clusters ($y^{(i)}$'s) that minimizes the cost (distortion) function J

  - Then, it assumes the cluster assignments ($y^{(i)}$'s) are known and finds the centroids $\boldsymbol{\mu}$'s that minimize J

- This method for minimizing a cost function is called **block coordinate decent**

Next Lecture (27.04.2026)

- Principle Component Analysis (PCA)

Thank you!